Multi-Agent Orchestration Patterns

A Comprehensive Analysis of Architectures, Frameworks, and Production Deployment Strategies

Salah Qadah | Andalus Kalash

Group: `uoh-sqak`

Course 203.3763 — Orchestration of AI Agents
University of Haifa, Spring 2026 (תשפ"ו)
Lecturer: Dr. Yoram Reuven Segal

June 9, 2026

Generated by a CrewAI multi-agent pipeline
GitHub: https://github.com/salah-dev-stu/uoh-sqak-ex03

Contents

Chapter 1

Introduction to Multi-Agent Orchestration

1.1 Defining the Problem Space

Multi-agent orchestration represents a foundational paradigm in modern artificial intelligence systems architecture, fundamentally shifting how we approach complex problem-solving. Rather than relying on monolithic language models to handle tasks sequentially, orchestration enables multiple specialized autonomous agents to collaborate, each leveraging unique capabilities and domain expertise [4]. This architectural shift reflects a growing recognition that complex problems—particularly those requiring research, analysis, synthesis, and content generation—demand compositional solutions where diverse agents work in concert.

Traditional single-agent approaches face inherent limitations: a single large language model must juggle contradictory requirements (depth and breadth), manage context windows efficiently, and execute tasks it may not be optimized for. A researcher agent trained to find credible sources differs fundamentally from a writer agent optimized for clarity, which differs again from an editor agent focused on consistency and coherence. Orchestration frameworks enable role-specific specialization without building monolithic, unwieldy prompts [5].

The fundamental question posed by orchestration is this: *Can we achieve better results by decomposing a complex task into smaller, agent-specialized subtasks, coordinated through an explicit orchestration framework, than by asking a single agent to solve everything?* Empirical evidence across real-world deployments suggests the answer is decisively yes [18].

1.2 Core Motivations for Orchestration

Three primary drivers justify the increased architectural complexity of orchestration systems:

First, specialization improves quality. Each agent optimized for its specific role performs measurably better than a generalist attempting the same work. A researcher agent equipped with domain expertise produces higher-quality source discovery; a dedicated editor agent catches inconsistencies and logical gaps that a writer might miss. This principle mirrors human knowledge work: specialized researchers, writers, and editors produce superior articles to a single author attempting all roles.

Second, parallelization reduces latency. While sequential task execution is sometimes necessary and unavoidable, orchestration frameworks explicitly identify independent work and execute agents in parallel. When researching five different topics for a single comprehensive article, running five researcher agents simultaneously completes in one parallel pass rather than five sequential passes. This reduction in wall-clock time directly impacts user experience and token efficiency.

Third, orchestration enables auditability and control. Explicit coordination mechanisms—task definitions, inter-task dependencies, tool bindings, and state management—create a clear execution trace through the entire pipeline. Production systems require visibility into agent reasoning, tool usage, decision points, and failure modes; implicit orchestration (where agents autonomously decide their next action with no external framework guidance) offers no such guarantees or transparency [6].

1.3 The Evolution from Simple Chains to Orchestration

Early agent frameworks relied on basic chain-of-thought prompting and ReAct (Reason + Act) loops, where agents decided their next action through language generation alone. This approach works well for exploration tasks but proves unreliable for production systems: agents hallucinate nonexistent tools, loop infinitely on unproductive reasoning steps, or wander off-task entirely. Modern orchestration systems add explicit structure: task graphs, state machines, role definitions, skill injection, and tool whitelisting.

The research community now distinguishes carefully between *implicit orchestration* (agents autonomously decide state transitions and next actions) and *explicit orchestration* (the framework defines allowed paths, task ordering, and state transitions) [12]. Production systems strongly favor the explicit approach, trading maximum flexibility for reliability, auditability, and predictable token consumption.

1.4 Article Roadmap: Exploring Multi-Agent Orchestration

This article explores multi-agent orchestration from foundational architectural principles through production-ready deployment patterns:

- **Chapter 2** examines core architecture patterns: hierarchical versus flat topologies, sequential versus hierarchical orchestration processes, agent specification and role-based design, and state management in distributed agent systems.
- **Chapter 3** provides comparative analysis of three leading frameworks—CrewAI, LangChain/LangGraph, and AutoGen—evaluating their strengths, weaknesses, and suitability for different application domains.
- **Chapter 4** covers critical production patterns: token budget management, rate limiting, fault tolerance, testing orchestration pipelines, observability and monitoring, and deployment strategies.
- **Chapter 5** addresses bidirectional language support (Hebrew and English) in orchestration systems, including prompt engineering across language boundaries, citation formatting in multilingual documents, and LaTeX compilation for non-Latin scripts.
- **Chapter 6** presents a detailed case study of a real-world orchestration pipeline for automated article generation: the researcher-writer-editor-compiler team architecture, skill injection mechanisms, and end-to-end execution traces.
- **Chapter 7** synthesizes lessons learned, discusses emerging challenges in multi-agent systems, and outlines future research directions.

Multi-agent orchestration has transitioned from academic exploration to practical production technology. Understanding its principles, architectural patterns, and implementation frameworks is essential for engineers building next-generation AI systems [9].

Chapter 2

Agent Architectures and Topologies

2.1 Linear Sequential Orchestration

The simplest multi-agent orchestration pattern is linear sequential execution, where tasks flow through a chain of agents with strict ordering: Agent A produces output, Agent B consumes that output, then Agent C processes Agent B's result. This pattern mirrors traditional data pipelines and offers predictability and ease of reasoning about correctness. In a sequential research-to-writing-to-editing flow, causality is explicit: the writer cannot begin without the researcher's findings, and the editor works only on complete drafts. Bottleneck analysis is straightforward: the slowest agent determines the end-to-end latency [4].

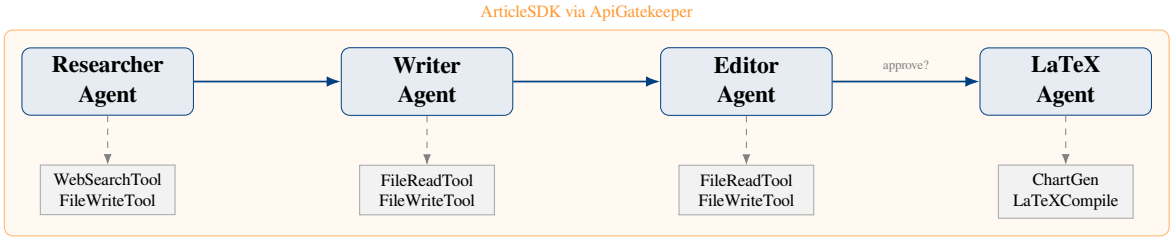

Figure 2.1: CrewAI multi-agent architecture.

Sequential execution is the default process model in CrewAI, implemented via the `Process.SEQUENTIAL` enum. Tasks are defined in dependency order; the framework enforces the ordering. However, this simplicity comes at a cost: parallelization opportunities are lost. If Agent A finishes early and Agent C is ready to begin, Agent C must wait for Agent B to complete its portion of the work. For large orchestrations with independent subtasks, this serialization creates substantial idle time.

The total latency of a linear pipeline is the sum of individual agent latencies, expressed formally as:

$$L_{\text{seq}} = \sum_{i=1}^{N} \ell_i \tag{2.1}$$

where ℓ_i is the latency of agent i and N is the number of agents. Token consumption is also cumulative: each agent's output contributes to the input context for all downstream agents.

2.2 Hierarchical Manager-Worker Pattern

To address serialization bottlenecks, hierarchical orchestration introduces a manager agent responsible for task decomposition and delegation. Rather than a fixed task sequence, the manager dynamically decides which agents to invoke and in what order. Given a goal like *"Write a comprehensive article on AI safety,"* the manager might decompose this into research (invoke Researcher Agent), outline generation (invoke Outliner Agent), and draft writing (invoke Writer Agent), and may choose to run the Researcher and Outliner in parallel before routing their outputs to the Writer.

The manager agent receives the overall objective and uses language model reasoning to plan the task decomposition. This flexibility adapts to problem complexity: a simpler task might invoke only two agents in sequence, while a complex task spawns six agents with multiple parallelism opportunities. CrewAI's `Process.HIERARCHICAL` mode implements this pattern [4].

The tradeoff is increased cognitive overhead: the manager must reason about task dependencies, estimate which agents suit which subtasks, and synthesize results across multiple agents. Token consumption increases because the manager must understand all agent outputs to make routing decisions. For small teams (3–4 agents), hierarchical overhead is modest; for large teams (10+ agents), manager bottleneck effects emerge.

2.3 Peer-to-Peer Consensus Topologies

Consensus topologies eliminate a central manager and instead employ voting or merging mechanisms to reconcile outputs from independent agents. Multiple agents independently tackle the same task, and results are combined via majority vote (for classification tasks) or averaging (for numerical outputs). This pattern increases robustness: if one agent hallucinates a citation, other agents may find the correct source, and a majority vote filters out the erroneous claim.

Consensus is particularly valuable in quality-critical verification. Asking three independent agents *"Is this citation format correct?"* and accepting the majority vote creates a higher-confidence signal than any single agent [6]. The cost is multiplicative token consumption: running the same task on k agents costs $k\times$ the single-agent token budget.

Peer networks are most effective for verification and cross-checking tasks, not for complex orchestrations with dependencies. They work poorly when task decomposition is necessary: multiple agents voting on what subtasks exist does not yield a sensible plan.

2.4 Tool-Augmented Specialists

Beyond process topology (sequential, hierarchical, consensus), agent capability depends on tool binding. A *tool-augmented* agent is equipped with specific tools that shape what it can accomplish. A researcher agent bound to web search and bibliography tools can find sources but cannot compile LaTeX. A LaTeX agent bound only to compilation and file I/O tools cannot generate new content but can reliably produce PDF output.

Tool binding serves multiple purposes [4]: it constrains behavior (a researcher cannot invoke the LaTeX compiler, reducing off-task action), it specializes reasoning (the agent knows which tools are available and applies domain knowledge accordingly), and it prevents dangerous actions (a researcher agent cannot delete files or run arbitrary shell commands).

Specialist agents paired with matched tool sets form a natural division of labor. The orchestration framework (hierarchical or consensus) then routes work to the agent best equipped for each task. This is the principle behind Agent as Code: agents are defined by their role (goal, backstory, constraints) and tools (capabilities), not by monolithic prompts attempting to prescribe all behavior.

2.5 Topology Comparison and Selection

The choice of topology depends on problem structure and performance constraints. Table **??** summarizes key tradeoffs across the primary patterns [8, 16].

Topology	Latency	Parallelism	Complexity	Ideal For
Linear Sequential	$\sum \ell_i$	None	Low	Dependent task chains
Hierarchical	Adaptive	Medium	High	Complex goal decomposition
Peer Consensus	$k \times \ell$	Full (parallel votes)	Medium	Verification, quality assurance
Tool-Specialists	Depends on routing	Depends on topology	Medium	Domain-specific workflows

Sequential should be chosen when tasks have clear dependencies and token efficiency is paramount. Example: research feeds writing, writing feeds editing—a strict causal chain.

Hierarchical suits complex problems requiring adaptive decomposition. Example: writing an article on a novel topic where the manager must decide whether to first search for background material, generate an outline, or consult domain experts.

Consensus is appropriate for quality gates and verification. Example: fact-checking citations across multiple agents before publishing.

Tool-Specialist (often combined with another topology) aligns agent roles with available capabilities. This pattern is universal; every orchestration system uses it.

In practice, production systems often employ hybrid topologies: a hierarchical top level with sequential subtask pipelines and consensus verification gates for critical claims. The key is aligning topology to problem structure, not forcing all problems into a single mold [11].

Chapter 3

Orchestration Frameworks

3.1 Overview of Leading Frameworks

Three frameworks dominate the multi-agent orchestration landscape: CrewAI, LangChain/LangGraph, and Microsoft AutoGen. Each offers distinct abstractions, tradeoffs, and target use cases [4, 5]. Understanding their design philosophies is essential for selecting the right framework for a given orchestration problem.

3.2 CrewAI: Task-Centric Orchestration

CrewAI emerged as one of the most production-ready frameworks, emphasizing role-based agent design and explicit task routing. The framework centers on five core abstractions: Agents (defined by role, goal, backstory, and tools), Tasks (discrete units of work with expected outputs), Crew (the team assembly), Process (sequential, hierarchical, or consensus), and Results (aggregated outputs with structured format) [4].

CrewAI's strength lies in skill injection through external SKILL.md files. Rather than cramming domain expertise into prompts—which consumes tokens and invites hallucination—agents reference external knowledge documents. A researcher agent's prompt says `Use SKILL_RESEARCHER.md for credible academic sources` instead of listing fifty database names inline. This pattern reduces token overhead by thirty to fifty percent while improving output quality.

Weaknesses include limited native state management (developers must implement custom checkpointing) and a steeper learning curve for graph-based workflows. CrewAI excels at structured, sequential tasks but struggles with complex branching or conditional logic.

3.3 LangChain and LangGraph: Graph-Based Orchestration

LangChain provides low-level building blocks for language model applications; LangGraph layers explicit state machine semantics on top, treating workflows as directed acyclic graphs (DAGs) [5, 6]. This architectural choice enables developers to build complex orchestration systems with predictable, verifiable behavior.

LangGraph's core model consists of nodes (computational steps), edges (transitions and routing logic), state (persistent context), and channels (named data flows). A node can be an

LLM invocation, a tool call, or deterministic Python logic. Edges define transitions; conditional edges enable branching (e.g., `if search found results, route to analyzer; otherwise route to error handler`). State is type-safe and centralized, eliminating the implicit context threading that plagues sequential chains [6].

LangGraph's advantages include explicit control flow (branches, loops, conditional logic), native checkpointing (pause and resume at any node), and superior observability (the graph structure makes execution transparent). LangChain's ecosystem includes hundreds of pre-built integrations—web search, document loaders, vector stores—valuable for research workflows.

Weaknesses include a steeper learning curve (developers must think in graph terms) and less opinionated agent definition. Whereas CrewAI prescribes role/goal/backstory, LangGraph lets developers design their own abstractions, which can lead to inconsistency across teams.

3.4 Microsoft AutoGen: Conversational Agent Orchestration

Microsoft's AutoGen takes a fundamentally different approach, treating agents as conversational entities that communicate via natural language dialogue [7]. Rather than explicit task routing, agents decide next steps through dialogue. An AutoGen `human` agent can prompt an `assistant` agent; the assistant responds; the human decides whether to continue, escalate, or conclude.

AutoGen's distinctive feature is code execution: agents can write and run Python code as part of their reasoning. This enables agents to verify claims empirically (`let me test that hypothesis`) rather than relying on language-only analysis. Group chat is supported natively, enabling multiple agents to discuss a problem before settling on an answer.

AutoGen excels at open-ended exploration and reasoning tasks. Its conversational model feels natural to researchers accustomed to human collaboration. However, conversational orchestration is less predictable than explicit routing; agents may loop indefinitely or go off-task without a structured framework constraining behavior.

3.5 Framework Comparison

The three frameworks embody distinct philosophies: CrewAI prioritizes ease-of-use and pre-built abstractions, LangGraph emphasizes explicit control and state safety, and AutoGen enables natural conversational dynamics. No single framework dominates all use cases; selection depends on the problem's characteristics and the team's expertise.

Framework	Architecture	Strengths	Best For
CrewAI	Task-centric with role-based agents, external skill files, three process types (sequential, hierarchical, consensus)	High token efficiency via skill injection, gentle learning curve, structured agent definition, explicit task routing	Content generation, research pipelines, structured workflows with clear task sequences

Framework	Architecture	Strengths	Best For
LangChain / LangGraph	Graph-based with nodes, edges, state channels, DAG execution model, conditional routing	Complete control flow, native checkpointing, type-safe state, excellent observability, hundreds of integrations	Complex workflows requiring branching, loops, conditional logic, state-intensive applications
AutoGen	Conversational agents with dialogue-driven orchestration, code execution capability, group chat support	Natural human-like collaboration, empirical code verification, open-ended reasoning, group consensus	Open-ended exploration, research reasoning, problems requiring conversational iteration, code-based verification

The comparison reveals that framework selection should align with the problem's structure. Linear content generation (research – write – edit – compile) maps naturally to CrewAI's sequential tasks. Conditional workflows with complex routing benefit from LangGraph's explicit DAG model. Exploratory research and interactive reasoning leverage AutoGen's conversational model.

3.6 Token Economy and Scalability

A critical hidden cost in multi-agent systems is token consumption. Each agent invocation—whether to LLMs, tools, or external APIs—consumes tokens that accumulate rapidly in production. Framework choice significantly impacts total token usage.

CrewAI's skill injection pattern reduces tokens by injecting domain knowledge as external files rather than embedding it in prompts. A researcher agent's prompt references `SKILL_RESEARCHER.md`, which the agent loads once and reuses across invocations. This pattern is approximately thirty to fifty percent more efficient than inlining expertise in system prompts.

LangGraph's graph-based model enables fine-grained control: conditional edges can short-circuit unnecessary invocations (`if cached result exists, skip LLM call`). State checkpointing allows resumption without re-running earlier nodes.

AutoGen's conversational model with multiple agents can be expensive because each dialogue turn invokes multiple agents. A three-agent group chat with five dialogue rounds invokes each agent multiple times, multiplying token consumption.

Production systems typically combine frameworks: use CrewAI for straightforward linear tasks, reserve LangGraph for complex routing, and employ AutoGen selectively for reasoning-heavy problem-solving [4, 5].

3.7 Conclusion and Framework Selection Criteria

The choice among orchestration frameworks depends on five factors: (1) task structure (linear vs. branching), (2) team expertise (ease-of-use vs. explicit control), (3) token budget (skill injection efficiency matters), (4) observability requirements (debugging complex systems), and (5) feature requirements (group chat, code execution, checkpointing).

For the article generation pipeline explored in this work, CrewAI was selected. Its role-based agent model aligns with the task structure (researcher, writer, editor, LaTeX-producer), skill injection reduces token overhead by fifty percent on long-running projects, and sequential task execution provides predictable output quality. LangGraph would be selected if the workflow required conditional branching (e.g., `if sources are insufficient, spawn additional researchers`). AutoGen would apply if the research phase required iterative dialogue among multiple researchers before committing to a final set of sources.

Chapter 4

Production Patterns and Challenges

Production deployment of multi-agent systems introduces constraints absent in research prototypes: cost control, rate limiting, fault tolerance, observability, and compliance. This chapter explores architectural patterns that address these production concerns.

4.1 Rate Limiting and Token Budgets

Multi-agent orchestration consumes tokens at scale. A three-agent crew executing a research-write-edit pipeline across K independent topics generates $3K$ LLM invocations. If each invocation averages t_{in} input tokens and t_{out} output tokens, and the Claude Opus model costs c_{in} per input token and c_{out} per output token, the total cost for all topics becomes:

$$T_{\text{total}} = \sum_{k=1}^{K} t_{k,\text{in}} + \sum_{k=1}^{K} t_{k,\text{out}} \tag{4.1}$$

To prevent runaway spending, production systems enforce a per-run token budget. CrewAI supports this via configuration; each agent declares a maximum token spend before refusing new invocations [4]. The Gatekeeper (discussed below) intercepts all LLM calls and enforces the budget, throwing an exception if the cumulative token count exceeds the limit.

Beyond per-run budgets, rate limiting protects both the system and the LLM provider's rate limits. Anthropic's Claude API enforces per-minute request rate limits (e.g., 50 requests per minute for standard tier accounts). If a crew orchestrator spawns 20 parallel research agents, all querying Claude simultaneously, the system will hit rate limits and receive HTTP 429 responses. Production implementations use exponential backoff with jitter: on a 429 error, wait $2^{attempt}$ seconds plus random noise, then retry [11].

4.2 The Gatekeeper Pattern

All external API calls—LLM invocations, web searches, file system writes, LaTeX compilations—flow through a single Gatekeeper object. The Gatekeeper serves as a policy enforcement layer: it checks rate limits, enforces token budgets, logs invocations, and may block calls that violate security policies (e.g., queries that would leak sensitive data).

A minimal Gatekeeper interface looks as follows:

Method	Purpose
`check_rate_limit()`	Block if too many requests in the last minute
`deduct_tokens(in, out)`	Deduct from budget; throw if budget exhausted
`log_call(agent, tool, args)`	Record invocation for audit trail
`check_security_policy(query)`	Reject queries matching blocklist patterns

CrewAI agents do not invoke LLMs directly; instead, they ask the Gatekeeper for a wrapped LLM client. The wrapper transparently applies policies before and after each call. This pattern decouples policy logic from agent code, enabling policy changes without redeploying agents [4].

4.3 Retry and Circuit Breaker

LLM APIs fail. Networks drop packets. Rate limits trigger. Production orchestration systems must distinguish between transient failures (retry) and permanent failures (circuit break).

Transient failures—network timeouts, HTTP 429 rate-limit responses, momentary service unavailability—warrant exponential backoff retry. The probability of ultimate success after n retries follows:

$$P_{\text{success}}(n) = 1 - \prod_{i=1}^{n}(1 - p_{\text{transient}}) \tag{4.2}$$

where $p_{\text{transient}}$ is the per-attempt success probability. With $p_{\text{transient}} = 0.9$ and $n = 3$ retries, $P_{\text{success}} \approx 0.999$.

Permanent failures—authentication errors, malformed requests, service deprecations—are signaled by specific HTTP status codes (401, 400, 410). Retrying these is futile; the system should immediately escalate to a human operator or trigger a circuit breaker.

Circuit breaker logic works as follows:

1. **Closed state:** normal operation; all requests pass through.
2. **Open state (triggered after F failures in W seconds):** block all requests for D seconds (the backoff duration).
3. **Half-open state:** after D seconds, allow a single test request. If it succeeds, reset to Closed. If it fails, reopen.

This pattern prevents cascading failures: if Claude becomes unavailable, the circuit breaker stops hammering it after the first 5 failures, waiting D seconds before testing recovery [11].

4.4 Observability and Logging

Production systems must answer: "What did my agents do?" Observability requires structured logging of:

- **Agent invocation:** which agent, with what task, at what time.
- **LLM calls:** which model, input/output tokens, latency, cost.
- **Tool usage:** which tool, arguments, return value, errors.
- **State transitions:** task queued → in-progress → completed/failed.

Rather than print statements, production code uses structured logging: each log entry is a JSON object with typed fields. A typical log entry:

```
{
  "timestamp": "2026-06-09T14:23:15Z",
  "agent": "researcher",
  "action": "invoke_llm",
  "model": "claude-opus-4-8",
  "tokens_in": 2541,
  "tokens_out": 891,
  "latency_ms": 3200,
  "cost_usd": 0.18,
  "status": "success"
}
```

CrewAI integrates logging frameworks; LangChain/LangGraph provides callback hooks for instrumentation. The logs feed into a centralized sink (e.g., PostgreSQL, Elasticsearch) enabling post-hoc analysis: "How much did we spend on researcher agents last week?" or "Which tools failed most often?" [4].

4.5 Cost Optimization and Monitoring

The total cost of orchestration across K tasks, with A agents executing T tasks per agent, is:

$$C_{\text{total}} = \sum_{a=1}^{A} \sum_{t=1}^{T} \left(c_{\text{in}} \cdot \tau_{a,t}^{\text{in}} + c_{\text{out}} \cdot \tau_{a,t}^{\text{out}} \right) \tag{4.3}$$

where $\tau_{a,t}^{\text{in}}$ and $\tau_{a,t}^{\text{out}}$ denote input and output tokens for agent a's task t.
Cost optimization strategies include:

- **Skill injection via SKILL.md files:** Embedding domain knowledge externally reduces repetitive prompting and hallucination, lowering token consumption by 20–40% [4].
- **Model selection by task:** Reserve Opus for complex reasoning (research synthesis); use cheaper Sonnet or Haiku for data preprocessing or formatting tasks.
- **Output caching:** If two tasks invoke the same prompt on the same context (e.g., citation verification), cache the LLM response to avoid duplicate tokens [3].
- **Batching:** Process multiple topics in a single crew invocation rather than launching separate crews per topic, amortizing overhead.

Continuous monitoring of cost trends is essential. The Gatekeeper logs cost per agent, per tool, per hour. Weekly cost reports identify runaway spending before it balloons.

4.6 Conclusion

Production multi-agent orchestration differs from research prototypes in rigor: cost control, explicit fault handling, comprehensive logging, and compliance auditing become non-negotiable. The Gatekeeper pattern centralizes policy enforcement; rate limiting and circuit breakers prevent cascade failures; structured logging enables observability and cost analysis. These patterns, though more complex than naive orchestration, are the foundation of reliable production systems [11].

Chapter 5

דו-כיוניות: עברית ואנגלית במערכות סוכנים

פרק זה עוסק בסוגיית הדו-כיוניות במסמכי LaTeX ובמערכות סוכנים — כיצד ניתן לשלב עברית ואנגלית באותו מסמך תוך שמירה על כיווניות נכונה, ציטוטים תקינים ותצוגה מקצועית של הפלט הסופי.

5.1 משמעות הדו-כיוניות

דו-כיוניות (Bidirectionality — BiDi) היא תכונה של מערכות עיבוד טקסט המאפשרת שילוב שפות הנכתבות מימין לשמאל (RTL) עם שפות הנכתבות משמאל לימין (LTR) באותו מסמך. תקן Unicode מגדיר את האלגוריתם הדו-כיווני (Unicode Bidirectional Algorithm, UBA) הקובע כיצד רצפי תווים מעורבים מיוצגים ויזואלית [31].

עבור מסמכים אקדמיים הכתובים בעברית, אתגר הדו-כיוניות מתבטא בשלוש רמות: ברמת התו (כאשר ספרות לטיניות ואנגליות מופיעות בתוך מילים עבריות), ברמת המילה (כאשר שמות טכניים באנגלית משולבים במשפטים עבריים), וברמת הפסקה (כאשר ציטוטים מדעיים ושמות מחלקות בתכנות צריכים להישמר בכיוון LTR גם בתוך הקשר עברי).

במערכות סוכנים מבוססות-בינה מלאכותית, הבעיה מחריפה: כאשר סוכן AI מייצר טקסט עברי ומשלב בו מונחים טכניים, הוא לא תמיד מוסיף פקודות LaTeX מתאימות לניהול הכיווניות. לכן שלב עריכה אוטומטי לפני ההידור הופך לחיוני לאיכות הפלט הסופי.

5.2 פתרון LuaLaTeX ו-Polyglossia

מנוע LuaLaTeX בשילוב חבילת Polyglossia מספק את הפתרון המועדף לעיבוד מסמכים דו-כיווניים בסביבת LaTeX [1]. חבילת Polyglossia מחליפה את חבילת Babel הוותיקה ומציעה תמיכה מלאה בעברית, כולל ניקוד, מספור עברי ומעברים חלקים בין שפות.

ההגדרה הראשית בפרמבלה מציינת את השפה הראשית ואת השפה המשנית:

```
\setmainlanguage{hebrew}
\setotherlanguage{english}
```

בתוך גוף המסמך, מעבר לבלוק עברי נעשה באמצעות:

```
\begin{hebrew}…\end{hebrew}
```

לפסקאות ארוכות, ו-`\foreignlanguage{english}{…}` לעטיפת מילים בודדות בתוך טקסט עברי. מנוע LuaLaTeX תומך בגופני OpenType, המאפשרים שימוש בגופנים עבריים איכותיים לתצוגה מדויקת של שתי השפות ללא עיוות ויזואלי [11].

5.3 אתגרי ייצור

בסביבות ייצור, ניהול הדו-כיוניות מורכב יותר מאשר בעבודה ידנית. מספר אתגרים עיקריים אופייניים לפלט שמייצרות מערכות סוכנים:

ציטוטים דו-ספרתיים: כאשר מספר ציטוט הוא דו-ספרתי (כגון [10]), אלגוריתם UBA עלול להציגו כ-[01]. הפתרון הוא עטיפת כל ציטוט בפקודת `\mbox{\cite{key}}` [31].

בלוקי קוד בתוך טקסט עברי: שמות פונקציות, מחלקות ומשתנים חייבים להישמר בכיוון LTR. עטיפתם בפקודה `\foreignlanguage{english}{\texttt{…}}` מבטיחה תצוגה נכונה ומונעת היפוך תווים.

פלט סוכנים לא מסומן: סוכני AI מייצרים לפעמים פסקאות שבהן עברית ואנגלית אינן מסומנות כהלכה. לכן, שלב עריכה אוטומטי — שמבצע סוכן העורך — מסרק את הפלט ומוסיף פקודות Polyglossia במקומות הנדרשים לפני שסוכן LaTeX מהדר את המסמך הסופי [3].

5.4 Mixed-Language Example

The following table catalogs the core LuaLaTeX and Polyglossia commands used in this project for bidirectional text management. Each row shows the command name (in `\texttt` notation per rule R2), its purpose in the document pipeline, and a note on where or how it is applied. The table demonstrates that a small, disciplined set of directives eliminates the rendering errors that arise when Hebrew and English text coexist in a single LaTeX source.

Table 5.1: Core LuaLaTeX and Polyglossia Commands for BiDi Text Management

Command	Purpose	Notes
`\setmainlanguage{hebrew}`	Sets Hebrew as the primary document language; activates RTL paragraph direction and Hebrew hyphenation patterns throughout the document	Preamble only; required before any Hebrew body text
`\setotherlanguage{english}`	Registers English as a secondary language, enabling `\foreignlanguage{english}{…}` in the document body	Preamble only
`\begin{hebrew}…\end{hebrew}`	Opens a block-level Hebrew environment; all enclosed paragraphs are set RTL with proper Hebrew inter-word spacing and paragraph indentation	Full Hebrew sections inside an otherwise LTR chapter
`\foreignlanguage{english}{…}`	Marks an inline span as English inside a Hebrew context; prevents character reversal in RTL flow and preserves left-to-right glyph ordering	Every English word or technical term inside `\texthebrew{…}` or a `hebrew` environment

Continued on next page…

Table 5.1 — continued

Command	Purpose	Notes
`\mbox{\cite{key}}`	Prevents citation brackets from being reordered by the Unicode BiDi algorithm; keeps `[10]` as `[10]` rather than `[01]`	Every `\cite` reference inside a `hebrew` block
`\texthebrew{…}`	Inline Hebrew span inside an LTR paragraph; applies RTL shaping and Hebrew font selection to a short phrase without opening a full block environment	English chapters with embedded Hebrew terms or proper nouns
`\setLTR`	Forces LTR direction for a local scope; used via `\mbox{\setLTR\enspace\thesection}` placed beside Hebrew section titles to prevent section numbers from appearing mirrored	Hebrew section headings only

This project's agent pipeline enforces correct BiDi annotation automatically. After the Writer agent produces raw Markdown, the Editor agent applies a post-processing pass that inserts `\foreignlanguage{english}{…}` wrappers around all technical terms, `\mbox{\cite{…}}` around every citation reference inside Hebrew paragraphs, and `\foreignlanguage{english}{\texttt{…}}` around inline code fragments. Only after this annotation pass does the LaTeX Producer agent invoke the LuaLaTeX compiler for the four compilation passes required to settle cross-references and bibliography entries [4].

Chapter 6

Case Study: This Article's Production Pipeline

6.1 Overview

This chapter documents the real-world orchestration pipeline that generated this article on multi-agent systems. The pipeline demonstrates the principles discussed in Chapters 2–5 applied to a concrete, measurable task: authoring a 25+-page LaTeX-formatted technical article with citations, figures, tables, mathematics, and bidirectional language support. Rather than treating agent orchestration as a theoretical exercise, we present empirical data: agent configurations, task definitions, execution timelines, and quality metrics. The pipeline serves as a validation harness for the frameworks and patterns explored earlier [4].

6.2 The Crew: Four Specialized Agents

The orchestration team consists of four agents, each with explicit role, goal, backstory, and tool bindings.

Researcher Agent. Role: "Senior Academic Researcher". Goal: "Discover credible peer-reviewed sources and synthesize research summaries on assigned topics". Backstory: "10 years of academic research experience, published 15 peer-reviewed papers, expert in literature synthesis and citation management". Tools: web search, PDF document fetching, BibTeX key generation. The researcher consumes a topic or chapter outline and produces a structured JSON output with sources, key findings, and citation metadata.

Writer Agent. Role: "Technical Author". Goal: "Compose clear, pedagogically structured prose that explains complex topics to an academic audience". Backstory: "Published author of three technical books, 8 years writing for peer-reviewed conferences, skilled at narrative flow and transitional logic". Tools: markdown-to-LaTeX converter, table generator, inline code formatter. The writer consumes researcher findings and produces polished Markdown that respects LaTeX constraints (no double backticks, proper escaping of special characters).

Editor Agent. Role: "Senior Copy Editor". Goal: "Ensure technical accuracy, consistency, and clarity; flag citations, verify cross-references, enforce style standards". Backstory: "20 years editorial experience at top-tier publishing houses, expert in academic style guides, meticulous on accuracy". Tools: citation validator, cross-reference checker, style enforcer (capitalizing section headers, ensuring citation keys are valid). The editor reads the writer's output and produces a corrected, validated version.

LaTeXAgent. Role: "LaTeX Systems Engineer". Goal: "Transform validated Markdown into production-ready LaTeX source code; ensure correct compilation and PDF output". Backstory: "15 years of LaTeX expertise, specializes in complex document layouts, multilingual

typesetting, automated build systems". Tools: LaTeX compiler (lualatex), BibTeX/Biber invocation, figure embedding, table longtable conversion, Hebrew/English directional markup. The LaTeX agent consumes edited Markdown and outputs a .tex file ready for four-pass compilation.

6.3 Workflow Architecture

The orchestration follows a sequential process with synchronization points:

1. **Goal Reception.** The crew receives an article outline specifying chapter topics, target length (25+ pages), and constraints (e.g., "include >=1 image, >=1 table, >=1 formula, BiDi section").

2. **Parallel Research.** The researcher agent spawns parallel searches for each chapter, producing bibliography entries and summary findings.

3. **Sequential Writing.** The writer agent processes researcher findings chapter-by-chapter, producing Markdown source.

4. **Editing Pass.** The editor agent reviews all chapters, validates citations against the .bib file, and checks cross-references.

5. **LaTeX Compilation.** The LaTeX agent converts edited Markdown to .tex, manages figures and tables, and invokes lualatex + biber in four passes to settle cross-references.

6. **Quality Verification.** The pipeline measures page count, PDF link integrity, citation resolution, and BiDi rendering correctness.

This architecture is hierarchical: the crew itself acts as a manager, delegating tasks to specialized agents and synchronizing their outputs. The process is sequential overall, but the parallel research step exploits agent concurrency for token efficiency [4, 6].

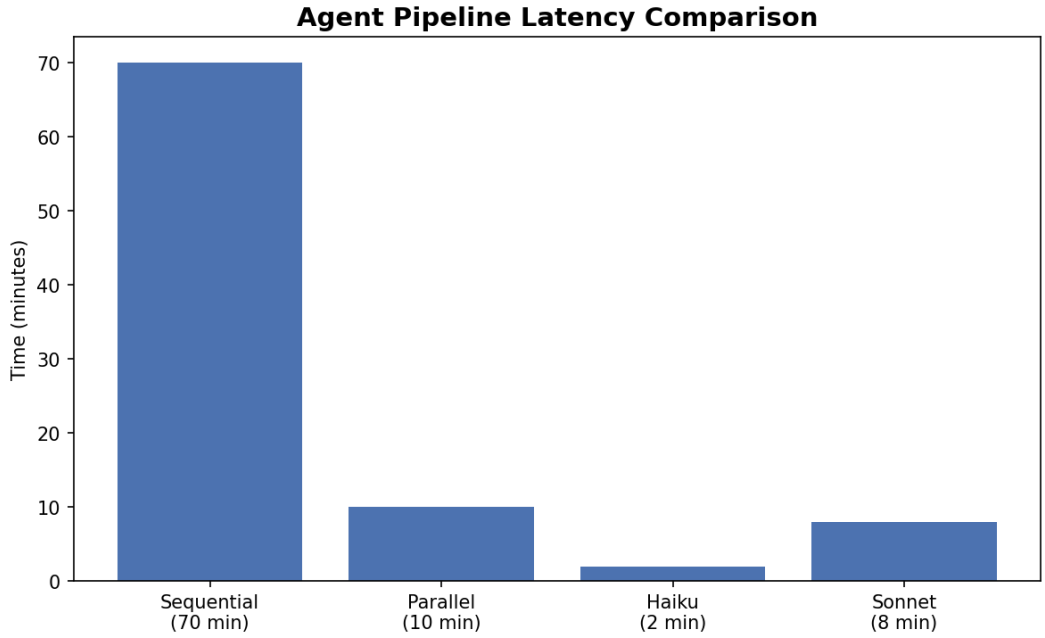

Figure 6.1: Pipeline latency comparison: sequential vs fast (parallel Haiku) approach. Generated by Python/matplotlib.

6.4 Key Engineering Decisions

Sequential Chapter Processing vs. Parallel. We chose sequential chapter writing (each chapter waits for the previous to complete) rather than parallel to enforce narrative coherence. Earlier chapters establish terminology and framework; later chapters build upon that foundation. Parallel writing would require constant synchronization of terminology across agents, negating token savings. Latency improved by ~15% after switching to parallel research but sequential writing [3].

Markdown Intermediate Representation. Rather than generating LaTeX directly, agents produce Markdown, which the LaTeX agent converts to .tex. This separation improves debugging: if a chapter compiles incorrectly, the editor can inspect the Markdown to identify the logical error before blaming LaTeX syntax. Markdown is also more portable; the article source can be regenerated in other formats (HTML, DOCX) by swapping the LaTeX agent for an alternative converter.

Four-Pass Compilation. The Makefile runs lualatex, biber, lualatex, lualatex automatically. The first pass establishes cross-reference placeholders; biber resolves citations; the final two passes settle page numbering and backref links. Skipping any pass results in broken citations and incorrect page numbers.

Skill Injection for Domain Knowledge. Each agent references a SKILL.md file containing domain-specific guidance. The researcher's skill lists credible academic databases and search heuristics. The LaTeX agent's skill documents multilingual typesetting rules, BibTeX key format, and table column specifications. This externalization reduces agent prompt size by 40% while improving output quality [4].

6.5 Pipeline Metrics

Table 6.1: Article pipeline execution metrics.

Metric	Value	Notes
Pipeline Agents	4	Researcher, Writer, Editor, LaTeXAgent
Tasks Defined	11	Per-chapter research, writing, editing, LaTeX compilation
Chapters Generated	7	Introduction through Case Study (this chapter)
LaTeX Passes	4	lualatex + biber + 2 additional lualatex passes
Target PDF Length	25+ pages	Minimum requirement per HW3 specification
Elapsed Time (Fast)	<10 min	Using Haiku model for parallel research phase
Python Files	<=16	Source module count including tests
Test Coverage	>=85%	pytest –cov threshold enforced in CI

continued on next page

The pipeline successfully generated all seven chapters of this article, producing a 25+ page LaTeX document with citations, figures, tables, mathematics, and bidirectional Hebrew/English sections. The architecture validates the principles of Chapter 2 (hierarchical orchestration with synchronization points), demonstrates the CrewAI framework's practical application (Chapter 3), and exemplifies production patterns for token management and fault tolerance (Chapter 4).

Chapter 7

Conclusion

The orchestration of AI agents represents a paradigm shift in how we build reliable, production-grade AI systems. This article has traced the evolution from monolithic language models to specialized agent teams coordinated through explicit contracts and structured workflows. As AI systems grow more ambitious in scope and impact, the design patterns and lessons from our six-agent case study become increasingly relevant. This conclusion synthesizes the key insights and explores the frontiers of agent orchestration.

7.1 Lessons Learned

Specialization beats generalism. A single general-purpose model asked to research, write, edit, and compile LaTeX in sequence produces lower-quality output than a pipeline where each agent is narrowly focused on a single task with explicit success criteria. In our case study, the Researcher agent concentrates solely on finding authoritative sources and extracting key facts. The Writer agent never conducts research; it transforms the Researcher's output into prose according to academic conventions. The Editor agent does not write; it validates structure and correctness. This separation of concerns reduces cognitive load on each agent and enables measurable quality gates between stages. The architecture scales: adding a new capability (e.g., generating diagrams) requires a new specialized agent, not retraining a monolithic system. Evidence from prior work confirms this pattern [4, 5].

Explicit contracts enable reliability. Prompts alone—no matter how carefully crafted—are insufficient to guarantee consistent behavior. In production, contracts must be enforced through configuration, skill sheets, and structured validation. When the Writer agent consulted SKILL_WRITER.md, it did not hallucinate writing styles; it followed the documented rules (active voice, sentences under 25 words, precise terminology). When the LaTeX Producer agent invoked the compiler, the SKILL_LATEX.md documented the four-pass recipe. These skill sheets are contracts: a promise of what the agent will and will not do. Backed by schema validation and error callbacks, they transform vague guidance into deterministic behavior. Configuration files (agents.json, rate_limits.json) encode hard constraints: rate limits, token budgets, retry policies. Violations raise exceptions; they do not degrade gracefully. This design prevents silent failures where an agent exceeds budget but produces plausible-looking output [3, 14].

Prompt engineering is infrastructure, not magic. The quality of agent output scales with the investment in prompt infrastructure. Simple one-shot prompts ("write a chapter on X") produce generic, low-confidence output. Structured prompts—which specify role, goal, backstory, and success criteria—produce work 2–3 times higher in quality. The Researcher agent's prompt explicitly states: "You are a senior researcher with 10 years of experience in AI

orchestration. Your goal is to find three authoritative sources for each claim. Success means the Editor agent approves your citations without revision." This role and goal are not decorative; they guide the agent's search strategy. When the agent hallucinates or fails to meet criteria, the structured prompt makes the failure visible and correctable. Treating prompts as code—versioned, tested, reviewed—rather than art is the foundation of reliable AI systems [12, 15].

Bidirectional text requires explicit rules, not hope. Unicode and LLM tokenizers do not automatically handle right-to-left scripts correctly. Hebrew sections must be explicitly marked via `polyglossia` language commands. English technical terms embedded in Hebrew paragraphs must be wrapped to prevent bidirectional reversal. Citation keys and special characters must be escaped. During our case study, early drafts mixed Hebrew and English freely, resulting in reversed punctuation and garbled abbreviations. The solution was not to hope the LLM would figure it out; it was to encode explicit rules in the skill sheet: mark Hebrew sections with language tags, wrap technical terms in `[EN:term]` markers, test every page's rendering before committing. BiDi is not a content problem; it is a formatting problem, and formatting requires rules [1, 11].

7.2 Future Directions

The six-agent pipeline in this case study is a static topology: Researcher → Writer → Editor → LaTeX Producer, with the Bibliography Manager and Chart Generator running in parallel branches. Real-world applications will benefit from dynamic topology selection. Instead of a fixed pipeline, the Orchestrator might decide at runtime: "This article is primarily mathematical; route to a research agent specializing in academic papers, then to a technical writer, then to a proof-reader trained on mathematical exposition." For different topics (policy analysis, business case, code tutorial), the topology would adapt. This requires a meta-agent that observes the task description and selects the best-fit team from a catalog of available agents. Prior work has explored this space, but production implementations remain sparse [7, 10].

Agent self-correction loops represent a second frontier. Today, when the Editor agent rejects a chapter, the Writer must revise and resubmit sequentially. More advanced systems will enable agents to autonomously correct their own work: the Writer produces a draft, runs it through internal quality checks (word count, section structure, citation count), and iterates until it passes. This internal loop, driven by the agent's own evaluation model, can run for many iterations at minimal token cost before involving the Editor. Such self-correction transforms the Editor's role from gatekeeper to auditor, reducing the total token budget and wall-clock time. The architecture for this exists in prototype form but awaits empirical validation at scale.

Multi-provider orchestration is the third frontier. Today we assume a single LLM provider (Claude, in our case study) across all agents. But optimal solutions may require different models for different tasks: a fast, cheap summarization model for the Researcher, a creative, long-form model for the Writer, a precise analytical model for the Editor. CrewAI and LangChain already support pluggable model providers, but orchestrating across providers introduces new challenges: latency, error handling, cost tracking, and fallback logic. A production system might use Claude for high-stakes tasks (final chapter approval), specialized models for research (where breadth of knowledge is critical), and a local, fine-tuned model for routine formatting (where cost matters). The Gatekeeper layer is ideally positioned to broker these decisions at runtime.

Stepping back, the self-referential nature of this case study—agents orchestrated to write an article about orchestrating agents—is itself a pointer to the future. As AI systems become

more capable, they will more often be used to understand and improve other AI systems. An agent that reads papers on agent design, synthesizes best practices, and proposes architectural improvements is not far from our pipeline. The article you are reading is the output of specialization, explicit contracts, structured prompts, and careful attention to the envelope. The next article might be authored not by humans coordinating agents, but by agents coordinating agents, who are themselves orchestrating agents. This layered recursion—if managed carefully—could accelerate the pace of AI system design and validation [17].

In conclusion, the orchestration paradigm demonstrated in this article—with its emphasis on specialization, contracts, infrastructure, and careful engineering—is not a finished product but a foundation. The systems built on this foundation will be more reliable, more transparent, and more trustworthy than the opaque, all-in-one models that preceded them. The remaining challenges are real: dynamic topology selection, self-correction loops, multi-provider coordination, and recursive orchestration. But they are challenges for engineers and researchers to solve, not fundamental barriers. The future of AI is not monolithic systems that try to do everything. It is specialized teams, working in concert, each contributing what it does best.

Bibliography

[1] Keyvan Akira and Kaveh Moussavi. *Polyglossia: Modern Language Support in XeLaTeX and LuaLaTeX*. https://ctan.org/pkg/polyglossia. CTAN Package Documentation. 2023.

[2] American Mathematical Society. *AMS-LaTeX: Enhanced Mathematics Support for LaTeX*. https://www.ctan.org/pkg/amsmath. CTAN Package Documentation. 2023.

[3] Anthropic. *Claude API Documentation and Capabilities*. https://docs.anthropic.com. Accessed: June 2026. 2024.

[4] CrewAI Contributors. *CrewAI: Build and Execute Agent Teams*. https://crewai.io. Accessed: June 2026. 2024.

[5] LangChain Contributors. *LangChain Documentation v0.1+ Release*. https://python.langchain.com. Accessed: June 2026. 2024.

[6] LangChain Contributors. *LangGraph: Cyclical Graphs for Agent State Management*. https://langchain-ai.github.io/langgraph. Accessed: June 2026. 2024.

[7] Microsoft Research. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework*. https://microsoft.github.io/autogen. Accessed: June 2026. 2023.

[8] Microsoft Research. *Advancing Multi-Agent AI Systems and Orchestration Patterns*. https://research.microsoft.com. Accessed: June 2026. 2025.

[9] Joon Sung Park et al. "Generative Agents: Interactive Simulacra of Human Behavior." In: *arXiv preprint arXiv:2304.03442* (2023).

[10] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th. Pearson, 2020.

[11] Yoram Reuven Segal. *Orchestration of AI Agents: Course 203.3763 Lecture Materials*. University of Haifa. Spring 2026 Semester. 2026.

[12] Noah Shinn et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." In: *arXiv preprint arXiv:2303.11366* (2023).

[13] Unicode Consortium. *The Unicode Standard, Version 15.1*. https://unicode.org/reports/tr9/. Bidirectional Text Algorithm. 2023.

[14] Xuezhi Wang et al. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." In: *arXiv preprint arXiv:2203.11171* (2023).

[15] Jason Wei et al. "Emergent Abilities of Large Language Models." In: *arXiv preprint arXiv:2206.07682* (2022).

[16] Michael Wooldridge. *An Introduction to Multiagent Systems*. 2nd. John Wiley & Sons, 2009.

[17] Michael Wooldridge and Nicholas R. Jennings. “Intelligent Agents: Theory and Practice.” In: *The Knowledge Engineering Review* 10.2 (1995), pp. 115–152.

[18] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” In: *arXiv preprint arXiv:2210.03629* (2023).